

LinkedList



Good

Morning

Agenda

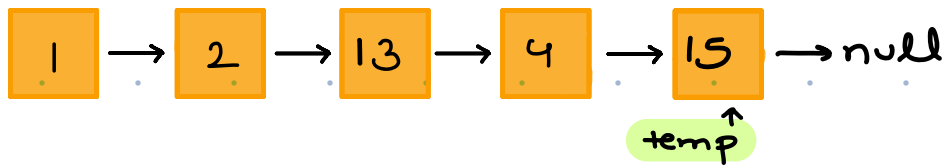
01. Search for a value
02. Insertion in LL
03. Deletion in LL
04. Reverse a LL
05. Deep copy of LL

A Linked List is a linear data structure where elements (called nodes) are stored in memory and connected using pointers.

```
class Node {  
    int data;  
    Node next;  
  
    Node(int x) {  
        data = x;  
        next = null;  
    }  
}
```

Ques: Given linkedlist, search for a value K .

Head



$K = 13 \rightarrow \text{true}$

$K = 7 \rightarrow \text{false}$

$K = 15 \rightarrow \text{true}$

boolean search (Node head , int K) {

Node temp = head;

TC : $O(N)$

SC : $O(1)$

while (temp != null) {

if (temp.data == K) return true;

temp = temp.next;

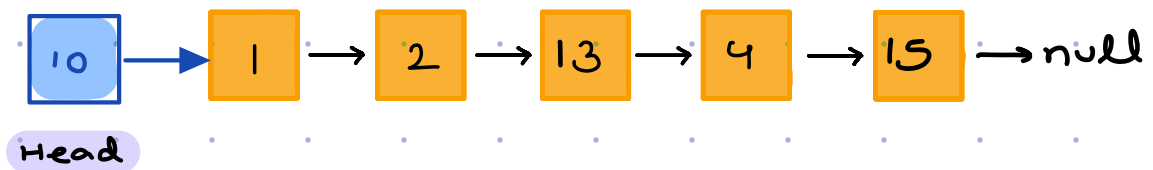
return false;

Ques 2 : Given a linkedlist, insert a new Node at the K^{th} position of Linkedlist

Head



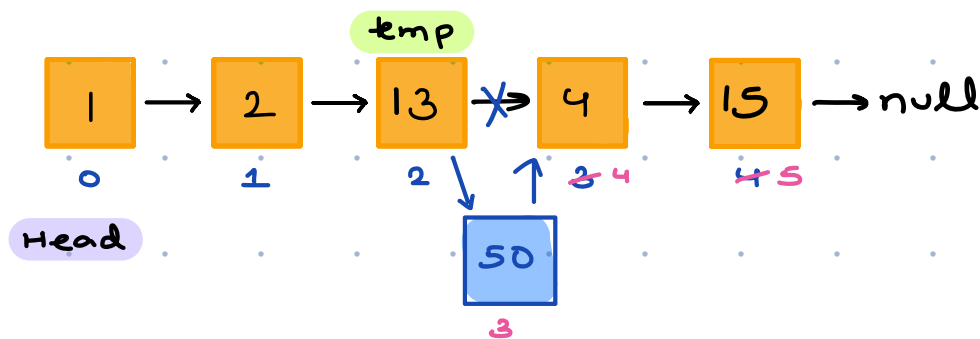
Case 1 : Data = 10 & $K = 0$



```

Node nn = new Node(10);
nn.next = head;
head = nn;
  
```

Case 2 : Data = 50 & K=3



$i = 1 < 3$
 $2 < 3$
 $3 < 3$ → stop

```
Node nn = new Node(10);
```

```
Node temp = head;
```

```
int i = 1;
```

```
while (i < k)
```

```

| temp = temp.next;
| i++;
|
  
```

```
nn.next = temp.next
```

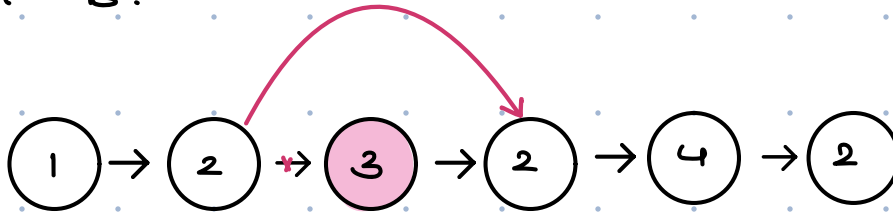
```
temp.next = nn;
```

```

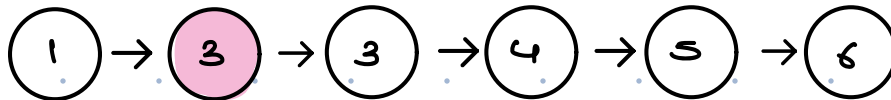
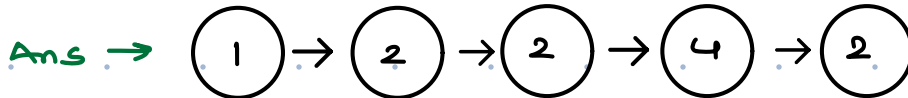
while (temp.next.data != k)
| temp = temp.next;
|
  
```

* Deletion in Linkedlist

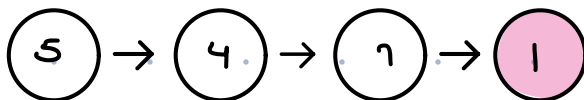
Delete the first occurrence of value x given in Linkedlist. If x is not present, leave Linkedlist as it is.



$x = 3$



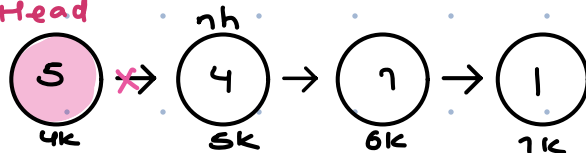
$x = 3$



$x = 1$



Head



$x = 5$

Head



Head = 4k

nh = 5k

head = 5k

```
Node deletefirst ( Node head, int x ) {
```

```
    if ( head == Null ) return head;
```

```
    if ( head.data == x )
```

```
        Node nh = head.next;
```

```
        head.next = Null;
```

```
        head = nh;
```

```
        return head;
```

```
Node temp = head;
```

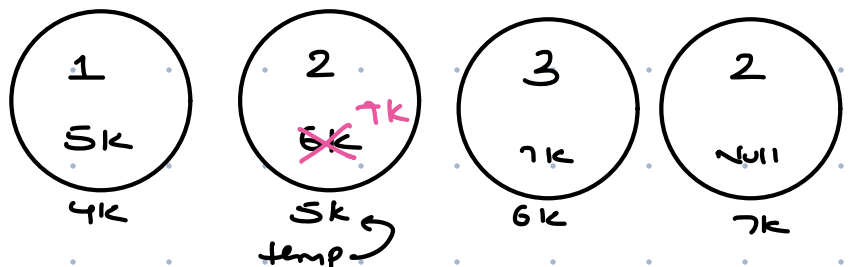
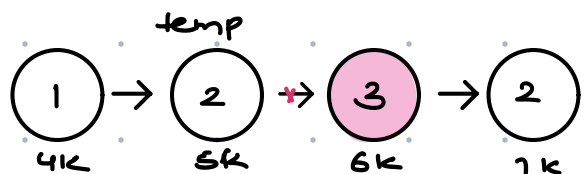
```
while ( temp != Null && temp.next != Null )
```

```
    if ( temp.next.data == x ) {
```

```
        temp.next = temp.next.next
```

```
        return head;
```

```
    temp = temp.next;
```



Scenario

OnePlus has a lineup of **N** mobile phones ready in their manufacturing line. It has detected a defect in one of their phone models during production.

They have decided to recall all phones of the defective model from their manufacturing line. Your task is to help **OnePlus** remove all defective phones from their production lineup efficiently.

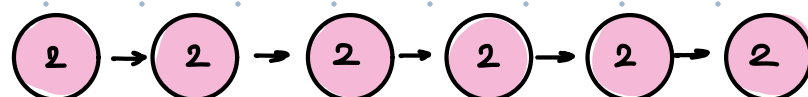
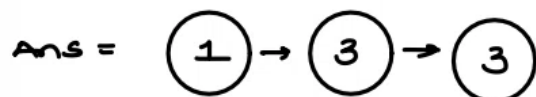
Problem

You are given a **linked list A** of **N** nodes where each node represents a specific **model type of a OnePlus mobile phone** in the manufacturing line. Each node contains an integer representing the model number of the phone. You will also be given an integer **B** which represents the model number of the defective phone that needs to be removed.

Your goal is to remove all **nodes** (phones) from the linked list that have the model number **B** and return the modified linked list representing the updated manufacturing line.



B = 2



B = 2

Ans = Null



```

Node deleteAll (Node head, int B)

```

```

while (head != Null && head.data == B)

```

```

    head = head.next;

```

```

Node temp = head;

```

```
while (temp != Null && temp.next != Null)
```

```
    if (temp.next.data == B) {
```

```
        temp.next = temp.next.next
```

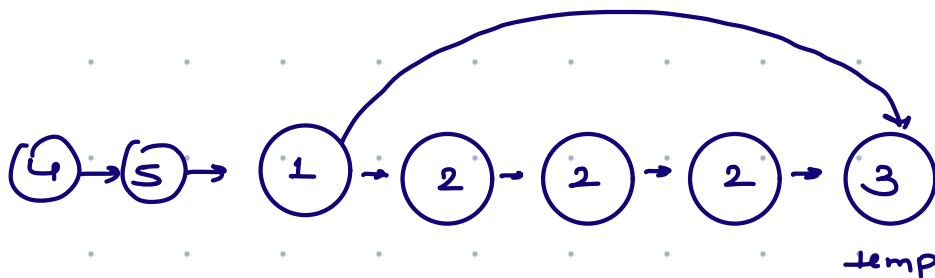
```
    } else {
```

```
        temp = temp.next;
```

```
    }
```

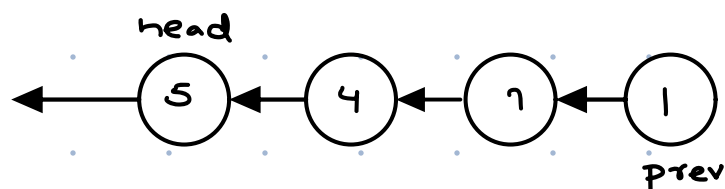
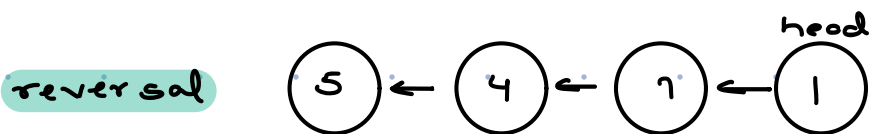
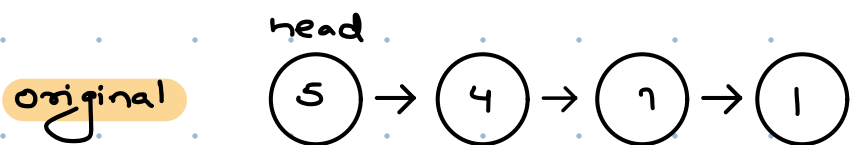
```
}
```

```
}
```



B = 2

* Reverse of LinkedList



Null

forw

curr

```
Node reverseLL (Node head) {
```

```
    Node prev = forw = Null;
```

```
    Node curr = head;
```

```
    while (curr != Null)
```

```
    {  
        forw = curr.next;
```

```
        curr.next = prev;
```

```
        prev = curr;
```

```
        curr = forw;
```

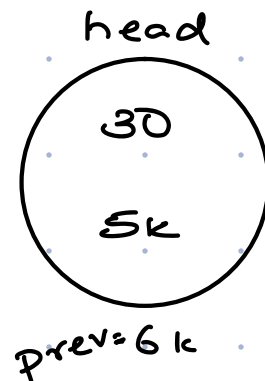
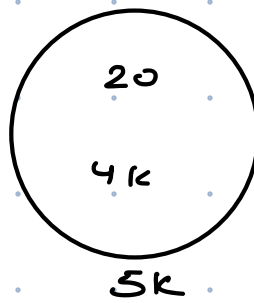
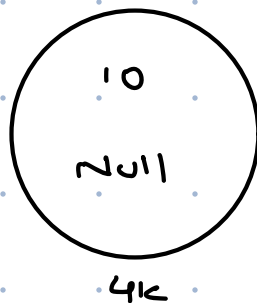
```
    }
```

```
    head = prev;
```

```
    return head;
```

TC: $O(N)$

SC: $O(1)$



forw = Null
curr =

clone Linkedlist

```
class Node {
```

```
    int data;
```

```
    Node next;    // pointing to next node
```

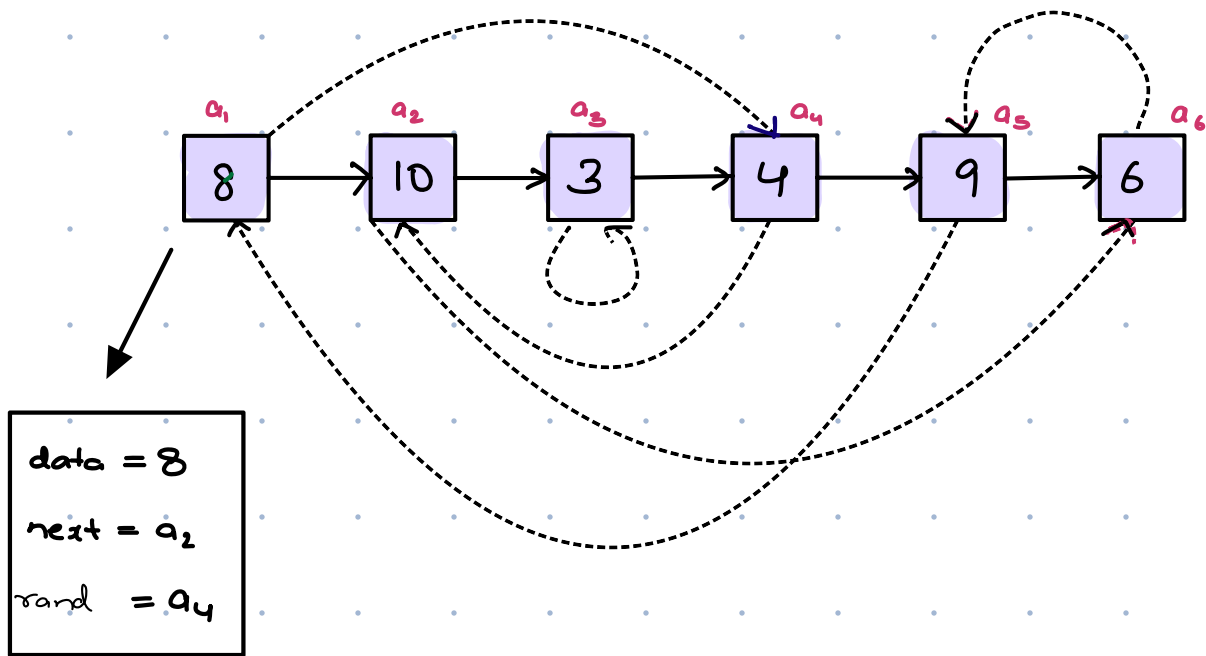
```
    Node rand    // pointing to any node in LL
```

```
}
```

Note :- Rand pointer is not null;

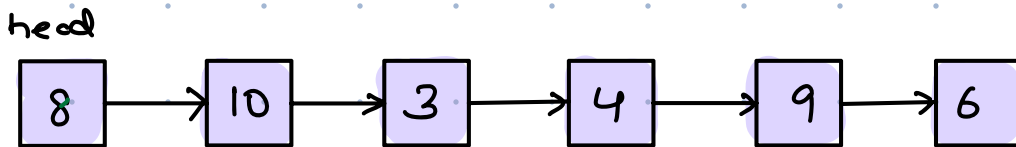
Given a LinkedList, create & return clone of it.

Expected SC: $O(1)$



Brute force →

01. Create a clone of LL just by using next pointer



```
Node nh = new Node(head.data)
```

```
Node temp = head.next;
```

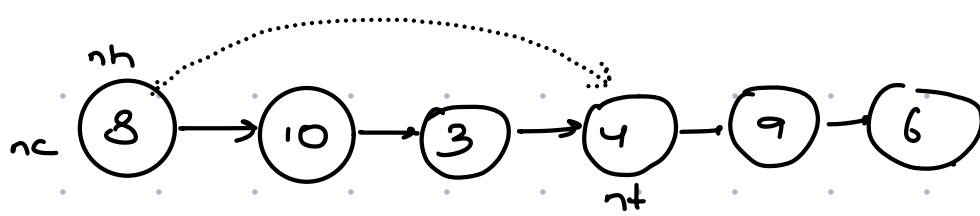
```
Node nt = nh;
```

```
while (temp != Null)
```

```
    nt.next = new Node(temp.data);
```

```
    nt = nt.next;    temp = temp.next;
```

```
,
```



02 To create random pointer,

⇒ extract rand pointer value from original linkedlist & then search for that value in cloned LL & create rand pointer.

Node curr = head

Node nc = nh;

while (curr != Null) {

int randval = curr->rand->data; // 4

while (nt->data != randval)

nt = nt->next;

nc->rand = nt;

curr = curr->next;

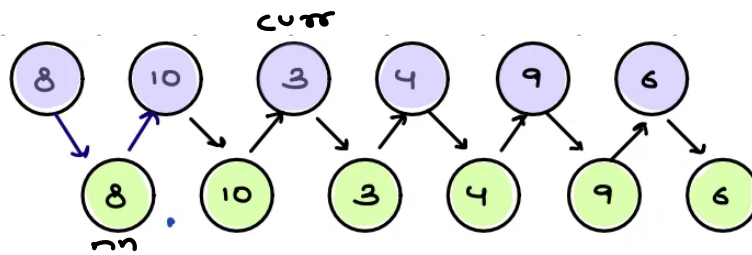
nc = nc->next;

TC: $O(N^2)$

SC: $O(1)$

* Optimal Idea

01. Interleave all the new nodes in between the old nodes



Node curr = head

while (curr != Null) {

Node nn = new Node(curr.data);

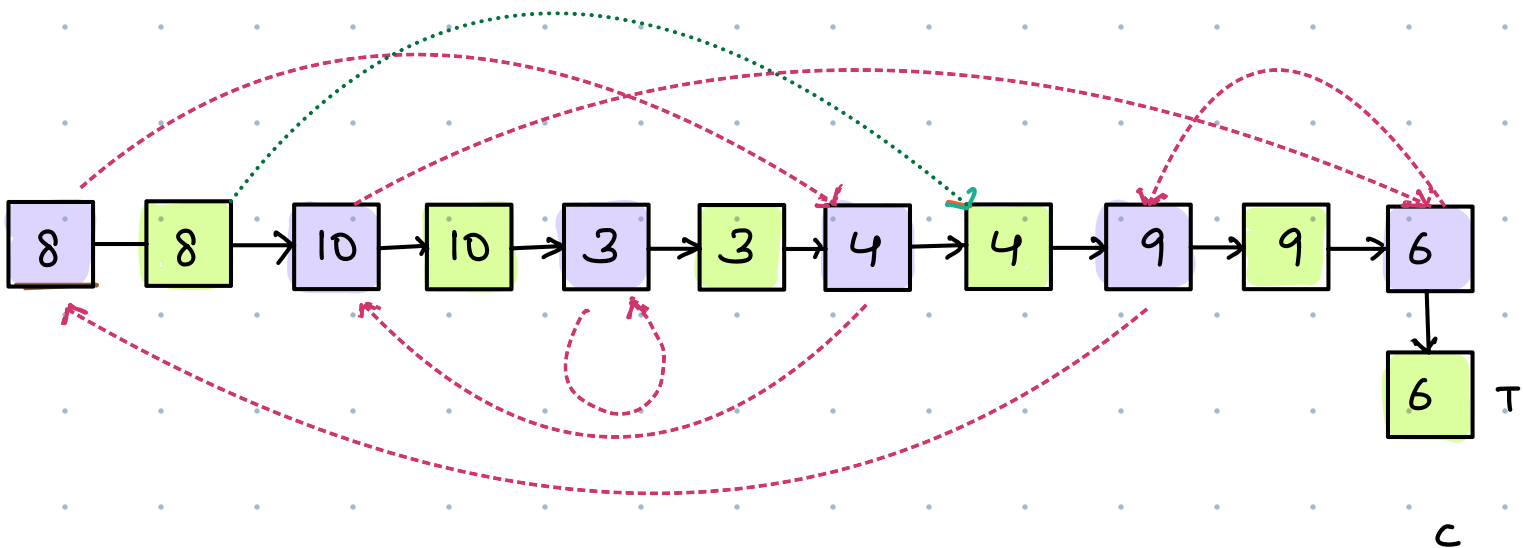
nn.next = curr.next;

curr.next = nn;

curr = nn.next;

TC: $O(N)$

* Populate random pointer using interleaving nodes technique



```
Node curr = head;
```

```
Node temp = curr.next;
```

```
while (curr != Null) {
```

```
    temp.next = curr.next.next;
```

```
    curr = curr.next.next;
```

```
    if (temp.next != Null)
```

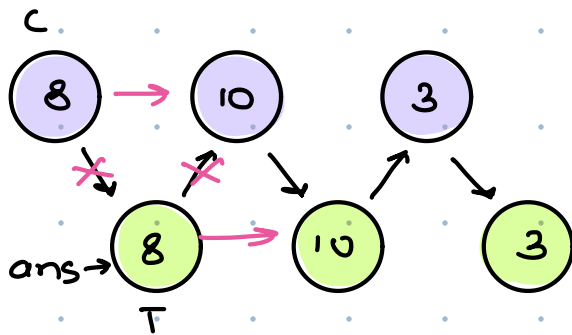
```
        temp = temp.next.next;
```

```
}
```

```
}
```

TC: $O(N)$

03. Detatch the old Linkedlist & new LL



```
Node curr = head;
```

```
Node temp = head.next;
```

```
Node ans = temp
```

```
while (curr != Null) {
```

```
    curr.next = curr.next.next;
```

```
    if (temp.next != Null) temp.next = temp.next.next;
```

TC: $O(N)$

```
curr = curr.next;
```

```
temp = temp.next;
```

```
return ans;
```

TC: $3N \approx O(N)$

SC: $O(1)$